

VERİ YAPILARI

Dr. Öğr. Üyesi Fatma AKALIN

İKİ YÖNLÜ BAĞLI LİSTE

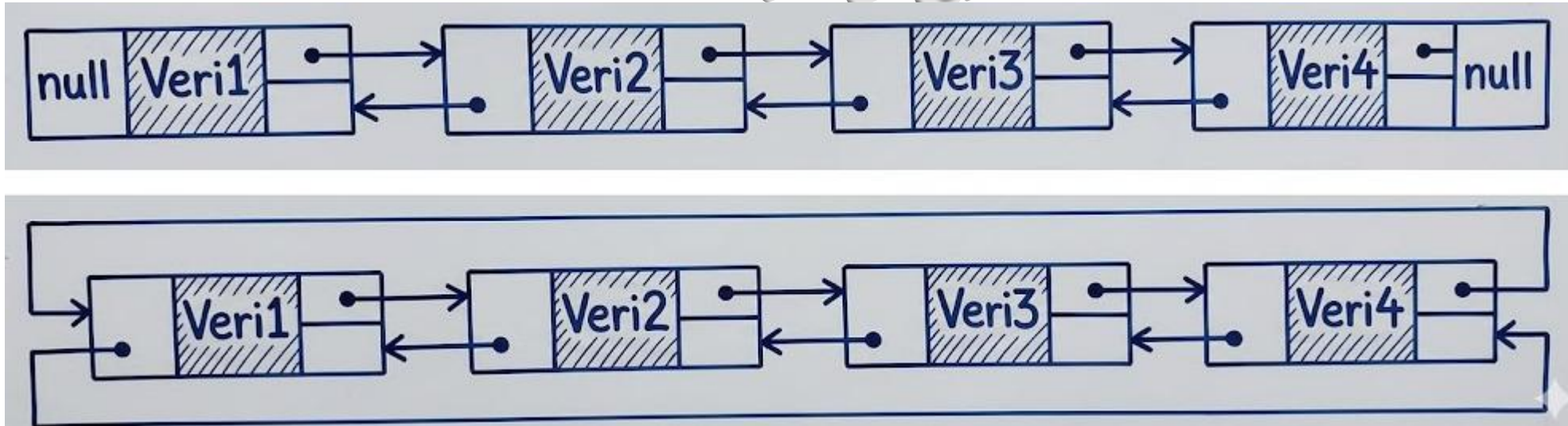
Dairesel bağlı listelerin kullanımı ile doğrusal bağlı listelerin bazı eksiklikleri giderildi. Fakat dairesel bağlı listelerinde bazı eksiklikleri bulunmaktadır. Örneğin dairesel liste yokken kesinlikle liste başından başlama zorunluluğu vardı. Fakat bu işlem için yaklaşık $n-1$ adet düğüme erişmek gerekmekte ve düğüm sayısının çok olması halinde bu işlem performansı olumsuz etkileyebilmektedir.

Eğer bir düğüm kendisinden sonra gelecek düğümün adres bilgisini içerdiği gibi kendisinden bir önceki düğümünde adres bilgisini içerseydi hedef düğümün kendinden önceki elemana erişmesi 1 adımda gerçekleşecekti.

Bu doğrultuda kendisinden önceki ve sonraki düğümlerin adreslerini saklamak üzere 2 adet işaretçi barındıran bazı liste yapılarına iki yönlü bağlı liste adı verilir.

Bağlı listelerdeki bu iki işaretçiyi düğümün kendisinden önceki düğümün adresini göstereni «sol» ve düğümün kendisinden sonra gelen düğümü adreslerini de «sağ» olarak isimlendireceğiz.

İki yönü bağlı listede doğrusal olarak tasarlanabileceği gibi dairesel olarak da tanımlanabilir. Aşağıdaki şekiller iki yönlü bağlı listenin **dairesel** ve **doğrusal** gösterimini tasvir etmektedir.



İKİ YÖNLÜ BAĞLI LİSTE

İki yönlü bağlı listede yapılacak temel işlemler eleman ekleme, eleman silme işlemleridir. Bunlar için kuracağınız algoritmalar listenin başına ekleme, listenin sonuna ekleme, listenin ortasına ekleme olarak değişebilir. Aynı biçimde eleman silme işlemleri de silinecek elemanın konumuna göre farklı algoritmalar ve dolayısıyla farklı güncellemelerle ifade edilebilir

Dr. Öğr. Üyesi Fatma AKALIN

```

1  #include <iostream>
2  using namespace std;
3  struct node{
4      string kullanıcıadi;
5      int parola;
6      node * next;
7      node * prev;
8  };
9  int main() {
10     struct node * uye1=new node(); //5 tane düğüm oluşturma siteye giriş yapacak
11     struct node * uye2=new node();
12     struct node * uye3=new node();
13     struct node * uye4=new node();
14     struct node * uye5=new node();
15
16     uye1->kullanıcıadi="a1";
17     uye2->kullanıcıadi="a2";
18     uye3->kullanıcıadi="a3";
19     uye4->kullanıcıadi="a4";
20
21     uye1->parola=111;
22     uye2->parola=222;
23     uye3->parola=333;
24     uye4->parola=444;
25
26     uye1->prev=NULL;
27     uye1->next=uye2;
28     uye2->prev=uye1;
29     uye2->next=uye3;
30     uye3->prev=uye2;
31     uye3->next=uye4;
32     uye4->prev=uye3;
33     uye4->next=NULL;
34
35     cout<<"1. düğümün kullanıcı adı ve şifresi : "<<uye1->kullanıcıadi<<" & "<<uye1->parola<<" "<<endl;
36     cout<<"2. düğümün kullanıcı adı ve şifresi : "<<uye2->kullanıcıadi<<" & "<<uye2->parola<<endl;
37     cout<<"3. düğümün kullanıcı adı ve şifresi : "<<uye3->kullanıcıadi<<" & "<<uye3->parola<<endl;
38     cout<<"4. düğümün kullanıcı adı ve şifresi : "<<uye4->kullanıcıadi<<" & "<<uye4->parola<<endl;
39     cout<<uye3->next->parola<<endl;
40     cout<<uye4->prev->parola<<endl;
41     return 0;
42 }
43

```

Tek tek düğüm oluşturup
çift yönlü bağlı bir listede
gösterme

```

cpp x ornek2.cpp x
#include <iostream>
using namespace std;
struct node{
    string kullanicadi;
    int parola;
    node * next;
    node * prev;
};
int main() {
    struct node * uye1=new node(); //5 tane düğüm oluşturma siteye giriş yapacak
    struct node * uye2=new node();
    struct node * uye3=new node();
    struct node * uye4=new node();
    struct node * uye5=new node();
    uye1->kullanicadi="a1";
    uye2->kullanicadi="a2";
    uye3->kullanicadi="a3";
    uye4->kullanicadi="a4";
    uye1->parola=111;
    uye2->parola=222;
    uye3->parola=333;
    uye4->parola=444;

    uye1->prev=NULL;
    uye1->next=uye2;
    uye2->prev=uye1;
    uye2->next=uye3;
    uye3->prev=uye2;
    uye3->next=uye4;
    uye4->prev=uye3;
    uye4->next=NULL;

    struct node * temp=uye1;

    int i=1;
    while(temp!=NULL) {
        cout<<i<<" düğümün kullanıcı adı ve şifresi : "<<temp->kullanicadi<<" & "<<temp->parola<<endl;
        temp=temp->next;
        i++;
    }
    cout<<"-----"<<uye3->next->parola<<"-----";
    cout<<"-----"<<uye4->prev->parola<<"-----";
    return 0;
}

```

Tek tek düğüm oluşturup
döngü vasıtasıyla çift
yönlü bağlı bir listede
gösterme


```

1.cpp x ornek2.cpp x ornek3.cpp x
#include <iostream>
using namespace std;

struct node{
    int kullaniciadi;
    int veri;
    node * next;
    node * prev;
};

struct node * basdugum=NULL;

struct node * basaekle(node * basdugum, int kullaniciadi, int veri){
    struct node * nnode=new node();
    nnode->kullaniciadi=kullaniciadi;
    nnode->veri=veri;
    if(basdugum==NULL){
        nnode->next=NULL;
        nnode->prev=NULL;
        basdugum=nnode;
    }
    else if(basdugum->next==NULL){
        nnode->prev=NULL;
        nnode->next=basdugum;
        basdugum->prev=nnode;
        basdugum->next=NULL;
        basdugum=nnode;
    }
    else{
        nnode->next=basdugum;
        basdugum->prev=nnode;
        nnode->prev=NULL;
        basdugum=nnode;
    }
    return basdugum;
}

void yazdir(node * basdugum){
    if(basdugum==NULL){
        cout<<"Yazdirilacak eleman yok";
    }
    node * iter=basdugum;

    while(iter!=NULL){
        cout<<"kullanici adi "<<iter->kullaniciadi<<" Veri "<<iter->veri<<endl;
        iter=iter->next;
    }
}

int main(){
    int kullaniciadi;
    int veri;
    node * root=NULL;
    root=basaekle(root,1,1);
    root=basaekle(root,2,2);
    root=basaekle(root,3,3);
    root=basaekle(root,4,4);
    yazdir(root);
}

```

Çift yönlü bağlı bir listede
başa eleman ekleyen
fonksiyonu gösterme

```
C:\Users\Fatma\Desktop\VERİYAPILARI\kitap\ciftyonlubagliliste>g++ -o ornek3 ornek3.cpp
```

```
C:\Users\Fatma\Desktop\VERİYAPILARI\kitap\ciftyonlubagliliste>ornek3
```

```
kullanici adi 4 Veri 4
```

```
kullanici adi 3 Veri 3
```

```
kullanici adi 2 Veri 2
```

```
kullanici adi 1 Veri 1
```

```

struct node * sonaekle(node * basdugum, int kullaniciadi, int veri){
    struct node * nnode=new node();
    nnode->kullaniciadi=kullaniciadi;
    nnode->veri=veri;
    if(basdugum==NULL){
        nnode->next=NULL;
        nnode->prev=NULL;
        basdugum=nnode;
    }
    else if(basdugum->next==NULL){
        basdugum->next=nnode;
        nnode->prev=basdugum;
        basdugum->prev=NULL;
        nnode->next=NULL;
    }
    else{
        node * iter=basdugum;
        while(iter->next!=NULL){
            iter=iter->next;
        }
        iter->next=nnode;
        nnode->prev=iter;
        nnode->next=NULL;
    }
    return basdugum;
}

```

Çift yönlü bağlı bir listede
sona eleman ekleyen
fonksiyonu gösterme

```

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\ciftyonlubagliliste>ornek4
kullanici adi 1 Veri 1
kullanici adi 2 Veri 2
kullanici adi 3 Veri 3
kullanici adi 4 Veri 4

```



```

struct node * arayaekle(struct node * basdugum, int kullaniciaadi, int veri, int aranandeger) {
    struct node * nnode = new node();
    nnode->kullaniciaadi = kullaniciaadi;
    nnode->veri = veri;
    if (basdugum == NULL) {
        cout << "Listede eleman yok";
    }
    if (basdugum->next == NULL && basdugum->veri == aranandeger) {
        nnode->prev = NULL;
        nnode->next = basdugum;
        basdugum->prev = nnode;
        basdugum->next = NULL;
        basdugum = nnode;
    }
    else {
        if (basdugum->veri == aranandeger) {
            nnode->next = basdugum;
            basdugum->prev = nnode;
            nnode->prev = NULL;
            basdugum = nnode;
        }
        else {
            node * newiter = basdugum;
            while (newiter->next->veri != aranandeger && newiter->next != NULL) {
                newiter = newiter->next;
                if (newiter->veri != aranandeger && newiter->next == NULL) {
                    cout << "Aranan deger bulunamadi!!--Lutfen var olan bir deger giriniz" << endl;
                    return basdugum;
                }
            }

            node * temp = newiter->next;
            newiter->next = nnode;
            nnode->prev = newiter;
            nnode->next = temp;
            temp->prev = nnode;
        }
    }
    return basdugum;
}

```

Çift yönlü bağlı bir listede
araya eleman ekleyen
fonksiyonu gösterme

```

int main() {
    cout << "Listedeki hangi kullaniciaadina  
sahip elemandan once araya eleman  
eklemek istersiniz?";
    cin >> aranan;
    root = arayaekle(root, 7, 7, aranan);
    yazdir(root);
}

```

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\ciftyonlubagliliste>g++ -o ornek5 ornek5.cpp

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\ciftyonlubagliliste>ornek5

kullanici adi 1 Veri 1

kullanici adi 2 Veri 2

kullanici adi 3 Veri 3

kullanici adi 4 Veri 4

Listedeki hangi kullaniciadina sahip elemandan once araya eleman eklemek istersiniz?3

kullanici adi 1 Veri 1

kullanici adi 2 Veri 2

kullanici adi 5 Veri 5

kullanici adi 3 Veri 3

kullanici adi 4 Veri 4

Listedeki hangi kullaniciadina sahip elemandan once araya eleman eklemek istersiniz?2

kullanici adi 1 Veri 1

kullanici adi 6 Veri 6

kullanici adi 2 Veri 2

kullanici adi 5 Veri 5

kullanici adi 3 Veri 3

kullanici adi 4 Veri 4

Listedeki hangi kullaniciadina sahip elemandan once araya eleman eklemek istersiniz?4

kullanici adi 1 Veri 1

kullanici adi 6 Veri 6

kullanici adi 2 Veri 2

kullanici adi 5 Veri 5

kullanici adi 3 Veri 3

kullanici adi 7 Veri 7

kullanici adi 4 Veri 4

```
struct node * basdugumusil(node * basdugum){
    if(basdugum==NULL){
        cout<<"Liste bos";
    }
    else if(basdugum->next==NULL){
        delete basdugum;
        basdugum=NULL;
    }
    else{
        node * temp=basdugum->next;
        temp->prev=basdugum;
        delete basdugum;
        basdugum=temp;
        basdugum->prev = NULL;
    }
    return basdugum;
}
```

Çift yönlü bağlı bir listede
baş düğümü silen
fonksiyonu gösterme

```
struct node * sondansil(node * basdugum){
    if(basdugum==NULL){
        cout<<"Silinecek eleman yok";
    }
    else if(basdugum->next==NULL){
        delete basdugum;
        basdugum=NULL;
    }
    else{
        node * iter=basdugum;
        while(iter->next->next!=NULL){
            iter=iter->next;
        }
        node * temp=iter->next;
        temp->next=NULL;
        temp->prev=iter;
        delete temp;
        iter->next=NULL;
    }
    return basdugum;
}
```

Çift yönlü bağlı bir listede
son düğümü silen
fonksiyonu gösterme

```

struct node * aradansil(struct node * basdugum,int aranandeger){
    struct node * nnode=new node();

    if(basdugum==NULL){
        cout<<"Listede eleman yok";
    }
    else if(basdugum->next==NULL && basdugum->veri==aranandeger){
        delete basdugum;
        basdugum=NULL;
    }
    else{
        if(basdugum->veri==aranandeger ){
            node * temp=basdugum->next;
            temp->prev=basdugum;
            delete basdugum;
            temp->prev=NULL;
            basdugum=temp;
        }
        else{
            node * iter=basdugum;
            while(iter->next!=NULL && iter->next->veri!=aranandeger){
                iter=iter->next;
            }
            if(iter->veri!=aranandeger && iter->next==NULL){
                cout<<"Aradiginiz eleamn bulunamadi";
                return basdugum;
            }
            else if(iter->next->veri==aranandeger && iter->next->next==NULL){
                node * temp=iter->next;
                temp->prev=iter;
                delete temp;
                iter->next=NULL;
            }
            else{
                node * stemp=iter->next->next;
                node * aranan=iter->next;
                delete aranan;
                iter->next=stemp;
                stemp->prev=iter;
            }
        }
    }
    return basdugum;
}
}

```

Çift yönlü bağlı bir listede
aradan düğümü silen
fonksiyonu gösterme

```

int main{
    cout<<"Listedeki hangi veriye sahip elemani silmek istersiniz?";
    cin>>aranan;
    root=aradansil(root,aranan);
    yazdir(root);
}

```


Çift Yönlü Doğrusal Bağlı Liste (DLL)

```
struct node {  
    std::string kullaniciadi;  
    int parola; // parola sayisal olarak tutuluyor  
    node* next;  
    node* prev;  
};
```

```
// 1. Parola toplamını güncelle  
void parolaGuncelleDLL(node* bas, int delta) {  
    node* iter = bas;  
    while (iter) {  
        iter->parola += delta;  
        iter = iter->next;  
    }  
}
```

while(iter) ifadesi, "iter (işaretçisi) boş olmadığı sürece (bir adresi gösterdiği sürece) devam et" demektir.

```
return (count == 0) ? 0 : toplam / count;
```

Koşul (count == 0): "Listede hiç eleman var mı?" diye sorar.

Soru İşareti (?): "Eğer bu koşul doğruysa (liste boşsa)..."

İlk Değer (0): "...sonuç olarak 0 döndür."

İki Nokta Üst Üste (:): "Eğer koşul yanlışsa (liste boş değilse, yani eleman varsa)..."

İkinci Değer (toplam / count): "...toplamı eleman sayısına böl ve ortalamayı döndür."

```
// 2. Parola ortalaması  
double parolaOrtDLL(node* bas) {  
    node* iter = bas;  
    double toplam = 0;  
    int count = 0;  
    while (iter) {  
        toplam += iter->parola;  
        count++;  
        iter = iter->next;  
    }  
    return (count == 0) ? 0 : toplam / count;  
}
```

```
// 3. Ters sırada yazdır
void printReverseDLL(node* bas) {
    if (!bas) return;
    node* iter = bas;
    while (iter->next) iter = iter->next; // sona git
    while (iter) {
        std::cout << iter->kullaniciadi << ": " << iter->parola << std::endl;
        iter = iter->prev;
    }
}
```

```
// 4. Çift parolaları yazdır
void printEvenDLL(node* bas) {
    node* iter = bas;
    while (iter) {
        if (iter->parola % 2 == 0)
            std::cout << iter->kullaniciadi << ": " << iter->parola << std::endl;
        iter = iter->next;
    }
}
```

```
// 6. Belirli uzunlukta isimleri yazdır
void printNameLengthDLL(node* bas, int length) {
    node* iter = bas;
    while (iter) {
        if (iter->kullaniciadi.length() == length)
            std::cout << iter->kullaniciadi << ": " << iter->parola << std::endl;
        iter = iter->next;
    }
}
```

```
// 5. Tek parolaları sil
node* removeOddDLL(node* bas) {
    node* iter = bas;
    while (iter) {
        node* nextNode = iter->next;
        if (iter->parola % 2 != 0) {
            if (iter->prev) iter->prev->next = iter->next;
            else bas = iter->next;
            if (iter->next) iter->next->prev = iter->prev;
            delete iter;
        }
        iter = nextNode;
    }
    return bas;
}
```

1. Hazırlık ve Döngü

node* nextNode = iter->next;: Bir düğümü sildiğinde, onun içindeki "bir sonraki düğümün adresi" bilgisi de kaybolur. Bu yüzden, silme işleminden önce bir sonraki düğümü nextNode değişkenine emanet ediyoruz ki yolumuzu kaybetmeyelim.

if (iter->parola % 2 != 0): Bu meşhur "tek sayı mı?" kontrolüdür. Parolanın 2'ye bölümünden kalan 0 değilse, o parola tektir ve silinmesi gerekir.

iter->prev **NULL değilse** (yani gerçekten bir önceki eleman varsa), bu bloğa gir.

2. Silme Ameliyatı (En Kritik Kısım)

Bir düğümü (Düğüm B diyelim) aradan çıkarmak için, önündeki (A) ve arkasındaki (C) düğümleri birbirine bağlaman gerekir.

- **if (iter->prev) iter->prev->next = iter->next;**: Eğer sildiğimiz düğümden önce bir düğüm varsa, o önceki düğümün next (sonraki) ucu artık sildiğimiz düğümü değil, bizim bir sonrakimizi (iter->next) gösterecek.
- **else bas = iter->next;**: Eğer sildiğimizden önce kimse yoksa, bu düğüm listenin başıdır. Artık yeni kafa (bas), bizim bir sonrakimiz olsun.
- **if (iter->next) iter->next->prev = iter->prev;**: Eğer sildiğimizden sonra bir düğüm varsa, onun prev (önceki) ucu artık bizi değil, bizim öncekimizi (iter->prev) gösterecek.

Temizlik ve İlerleme

- **delete iter;**: Düğümü hafızadan tamamen siler (çöp oluşmasını engeller).
- **iter = nextNode;**: Emanete aldığımız adrese giderek bir sonraki düğüme geçeriz.

ÇİFT YÖNLÜ DAİRESEL BAĞLI LİSTE

İki yönlü dairesel bağlı liste yapısında liste başı ya da liste sonu değişkenleri olmayacaktır. Dairesel bağlı tek yönde listede olduğu gibi listenin son düğümünün sağ işaretçisi null değil de listenin ilk elemanını adreslerse ve listenin ilk elemanının sol işaretçisi Listenin son elemanını adreslerse böylece iki yönlü ve dairesel bağlı liste yapısı elde edilmiş olacaktır.

Dr. Öğr. Üyesi Fatma AKALIN

```

ornek1.cpp x ornek2.cpp x ornek3.cpp x
1  #include <iostream>
2  using namespace std;
3  struct node{
4      string kullaniciadi;
5      int parola;
6      node * next;
7      node * prev;
8  };
9  int main() {
10     struct node * uye1=new node(); //5 tane düğüm oluşturma siteye giriş yapacak
11     struct node * uye2=new node();
12     struct node * uye3=new node();
13     struct node * uye4=new node();
14     struct node * uye5=new node();
15
16     uye1->kullaniciadi="a1";
17     uye2->kullaniciadi="a2";
18     uye3->kullaniciadi="a3";
19     uye4->kullaniciadi="a4";
20
21     uye1->parola=111;
22     uye2->parola=222;
23     uye3->parola=333;
24     uye4->parola=444;
25
26     uye1->prev=uye4;
27     uye1->next=uye2;
28     uye2->prev=uye1;
29     uye2->next=uye3;
30     uye3->prev=uye2;
31     uye3->next=uye4;
32     uye4->prev=uye3;
33     uye4->next=uye1;
34
35     cout<<"1. dugumun kullanıcı adı ve şifresi : "<<uye1->kullaniciadi<<" & "<<uye1->parola<<" "<<endl;
36     cout<<"2. dugumun kullanıcı adı ve şifresi : "<<uye2->kullaniciadi<<" & "<<uye2->parola<<endl;
37     cout<<"3. dugumun kullanıcı adı ve şifresi : "<<uye3->kullaniciadi<<" & "<<uye3->parola<<endl;
38     cout<<"4. dugumun kullanıcı adı ve şifresi : "<<uye4->kullaniciadi<<" & "<<uye4->parola<<endl;
39     cout<<uye3->next->parola<<endl;
40     cout<<uye4->prev->parola<<endl;
41     return 0;
42 }

```

Tek tek düğüm oluşturup
çift yönlü dairesel bağlı bir
listede gösterme


```

.cpp x ornek2.cpp x ornek3.cpp x
#include <iostream>
using namespace std;
struct node{
    string kullaniciadi;
    int parola;
    node * next;
    node * prev;
};
int main() {
    struct node * uye1=new node(); //5 tane düğüm oluşturma siteye giris yapacak
    struct node * uye2=new node();
    struct node * uye3=new node();
    struct node * uye4=new node();
    struct node * uye5=new node();
    uye1->kullaniciadi="a1";
    uye2->kullaniciadi="a2";
    uye3->kullaniciadi="a3";
    uye4->kullaniciadi="a4";
    uye1->parola=111;
    uye2->parola=222;
    uye3->parola=333;
    uye4->parola=444;
    uye1->prev=uye4;
    uye1->next=uye2;
    uye2->prev=uye1;
    uye2->next=uye3;
    uye3->prev=uye2;
    uye3->next=uye4;
    uye4->prev=uye3;
    uye4->next=uye1;
}

```

Tek tek düğüm oluşturup
döngü vasıtasıyla çift
yönlü dairesel bağlı bir
listede gösterme

YAZDIRMAK İÇİN

```

struct node *temp = uye1;
int i = 1;

```

```

do {
    cout << i << ". dugumun kullanci adi ve sifresi: "
    << temp->kullaniciadi << " & " << temp->parola << endl;

```

```

temp = temp->next;
i++;

```

```

} while(temp != uye1);

```

```

cpp x ornek2.cpp x ornek3.cpp x
#include <iostream>
using namespace std;

struct node{
    int kullaniciadi;
    int veri;
    node * next;
    node * prev;
};

struct node * basdugum=NULL;

struct node * basaekle(node * basdugum, int kullaniciadi, int veri){
    struct node * nnode=new node();
    nnode->kullaniciadi=kullaniciadi;
    nnode->veri=veri;
    if(basdugum==NULL){
        nnode->next=nnode;
        nnode->prev=nnode;
        basdugum=nnode;
    }
    else if(basdugum->next==basdugum){
        nnode->next=basdugum;
        basdugum->next=nnode;
        basdugum->prev=nnode;
        nnode->prev=basdugum;
        basdugum=nnode;
    }
    else{
        node * iter=basdugum;
        while(iter->next!=basdugum){
            iter=iter->next;
        }
        iter->next=nnode;
        nnode->next=basdugum;
        basdugum->prev=nnode;
        nnode->prev=iter;
        basdugum=nnode;
    }
    return basdugum;
}

```

```

void yazdir(node * basdugum){
    if(basdugum==NULL){
        cout<<"Yazdirilacak eleman yok";
    }
    node * iter=basdugum;
    while(iter->next!=basdugum){
        cout<<"kullanici adi "<<iter->kullaniciadi<<" Veri "<<iter->veri<<endl;
        iter=iter->next;
    }
    cout<<"kullanici adi "<<iter->kullaniciadi<<" Veri "<<iter->veri<<endl;
}

int main(){
    int kullaniciadi;
    int veri;
    node * root=NULL;
    root=basaekle(root,1,1);
    root=basaekle(root,2,2);
    root=basaekle(root,3,3);
    root=basaekle(root,4,4);
    yazdir(root);
}

```

Çift yönlü dairesel bağlı bir listede başa eleman ekleyen fonksiyonu gösterme

```

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\ciftyonludaireselbagliliste>g++ -o ornek3 ornek3.cpp
C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\ciftyonludaireselbagliliste>ornek3
kullanici adi 4 Veri 4
kullanici adi 3 Veri 3
kullanici adi 2 Veri 2
kullanici adi 1 Veri 1

```

```

struct node * sonaekle(node * basdugum, int kullaniciadi, int veri){
    struct node * nnode=new node();
    nnode->kullaniciadi=kullaniciadi;
    nnode->veri=veri;
    if(basdugum==NULL){
        nnode->next=nnode;
        nnode->prev=nnode;
        basdugum=nnode;
    }
    else if(basdugum->next==basdugum){
        basdugum->next=nnode;
        nnode->prev=basdugum;
        basdugum->prev=nnode;
        nnode->next=basdugum;
    }
    else{
        node * iter=basdugum;
        while(iter->next!=basdugum){
            iter=iter->next;
        }
        iter->next=nnode;
        nnode->prev=iter;
        nnode->next=basdugum;
        basdugum->prev=nnode;
    }
    return basdugum;
}

```

Çift yönlü dairesel bağlı bir
listede sona eleman
ekleyen fonksiyonu
gösterme

```

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\ciftyonludaireselbagliliste>g++ -o ornek4 ornek4.cpp
C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\ciftyonludaireselbagliliste>ornek4
kullanici adi 1 Veri 1
kullanici adi 2 Veri 2
kullanici adi 3 Veri 3
kullanici adi 4 Veri 4

```



```

struct node * arayaekle(node * basdugum,int kullaniciadi,int veri,int aranandeger){

    struct node * nnode = new node();
    nnode->kullaniciadi = kullaniciadi;
    nnode->veri = veri;

    if(basdugum==NULL){
        cout<<"Listede eleman yok";
        return basdugum;
    }

    // Tek elemanlı liste
    if(basdugum->next==basdugum && basdugum->veri==aranandeger){

        basdugum->next = nnode;
        basdugum->prev = nnode;
        nnode->next = basdugum;
        nnode->prev = basdugum;

        basdugum = nnode;

        return basdugum;
    }

    else{

        // Baş düğüm aranan ise (başa ekleme)
        if(basdugum->veri==aranandeger){

            node * son = basdugum->prev;

            nnode->next = basdugum;
            nnode->prev = son;
            son->next = nnode;
            basdugum->prev = nnode;

            basdugum = nnode;    // <<< yeni baş

            return basdugum;
        }

        else{

            node * newiter=basdugum;

            while(newiter->next!=basdugum && newiter->veri!=aranandeger){
                newiter=newiter->next;
            }

            // bulunamadiysa
            if(newiter->veri!=aranandeger){
                cout<<"Aranan deger bulunamadi!!"<<endl;
            }

            // araya ekleme
            node * temp=newiter->next;

            newiter->next=nnode;
            nnode->prev=newiter;
            nnode->next=temp;
            temp->prev=nnode;

        }

    }

    return basdugum;
}

```

```

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\ciftyonludaireselbagliliste>ornek5
kullanici adi 1 Veri 1
kullanici adi 2 Veri 2
kullanici adi 3 Veri 3
kullanici adi 4 Veri 4
Listedeki hangi kullaniciadina sahip elemandan once araya eleman eklemek istersiniz?3
kullanici adi 1 Veri 1
kullanici adi 2 Veri 2
kullanici adi 5 Veri 5
kullanici adi 3 Veri 3
kullanici adi 4 Veri 4
Listedeki hangi kullaniciadina sahip elemandan once araya eleman eklemek istersiniz?2
kullanici adi 1 Veri 1
kullanici adi 6 Veri 6
kullanici adi 2 Veri 2
kullanici adi 5 Veri 5
kullanici adi 3 Veri 3
kullanici adi 4 Veri 4
Listedeki hangi kullaniciadina sahip elemandan once araya eleman eklemek istersiniz?1
kullanici adi 7 Veri 7
kullanici adi 1 Veri 1
kullanici adi 6 Veri 6
kullanici adi 2 Veri 2
kullanici adi 5 Veri 5
kullanici adi 3 Veri 3
kullanici adi 4 Veri 4

```

Çift yönlü dairesel bağlı bir
listede araya eleman
ekleyen fonksiyonu
gösterme

```
struct node * basdugumusil(node * basdugum){  
    if(basdugum==NULL){  
        cout<<"Liste bos";  
    }  
    else if(basdugum->next==basdugum){  
        delete basdugum;  
        basdugum=NULL;  
    }  
    else{  
        node * iter=basdugum;  
        node * temp=basdugum->next;  
        while(iter->next!=basdugum){  
            iter=iter->next;  
        }  
        iter->next=temp;  
        temp->prev=iter;  
        delete basdugum;  
        basdugum=temp;  
    }  
    return basdugum;  
}
```

Çift yönlü dairesel bağlı bir
listede baş düğümü silen
fonksiyonu gösterme


```

struct node * sondansil(node * basdugum){
    if(basdugum==NULL){
        cout<<"Silinecek eleman yok";
    }
    else if(basdugum->next==basdugum){
        delete basdugum;
        basdugum=NULL;
    }
    else{
        node * iter=basdugum;
        while(iter->next->next!=basdugum){
            iter=iter->next;
        }
        node * temp=iter->next;
        iter->next = basdugum;
        basdugum->prev = iter;
        delete temp;
    }
    return basdugum;
}

```

Çift yönlü dairesel bağlı bir
listede son düğümü silen
fonksiyonu gösterme

- ✗ delete iter->next; → önerilmez
- ✓ önce pointer'ı sakla (temp)
- ✓ bağlantıları düzelt
- ✓ sonra sil

```

delete iter->next;
iter->next = basdugum;

```

iter->next **silindikten sonra artık geçersiz (dangling pointer)** olur.

- delete iter->next; dediğin anda o bellek **yok edildi**
- Ama iter->next hâlâ eski adresi tutar (çöp adres)
- Sonrasında işlem yapmaya çalışmak → **tanımsız davranış (undefined behavior)**

Ayrıca şu da eksik olur: ↻ basdugum->prev
güncellenmez. Yani liste bozulur

```

struct node * aradansil(node * basdugum,int aranandeger){
    if(basdugum==NULL){
        cout<<"Listede eleman yok";
        return basdugum;
    }

    else if(basdugum->next==basdugum && basdugum->veri==aranandeger){
        delete basdugum;
        basdugum=NULL;
    }

    else{
        if(basdugum->veri==aranandeger ){
            node * temp=basdugum->next;
            node * iter=basdugum->prev;

            iter->next=temp;
            temp->prev=iter;

            delete basdugum;
            basdugum=temp;
        }

        else{
            node * iter=basdugum;

            while(iter->next!=basdugum && iter->next->veri!=aranandeger){
                iter=iter->next;
            }

            if(iter->next==basdugum){
                cout<<"Aradiginiz eleman bulunamadi";
                return basdugum;
            }

            else if(iter->next->veri==aranandeger && iter->next->next==basdugum){

                node * temp=iter->next;

                iter->next=basdugum;
                basdugum->prev=iter;

                delete temp;
            }

            else{
                node * stemp=iter->next->next;
                node * aranan=iter->next;

                iter->next=stemp;
                stemp->prev=iter;

                delete aranan;
            }
        }

        return basdugum;
    }
}

```

```

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\ciftyonludaireselbagliliste>g++ -o ornek8 ornek8.cpp

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\ciftyonludaireselbagliliste>ornek8
kullanici adi 1 Veri 1
kullanici adi 2 Veri 2
kullanici adi 3 Veri 3
kullanici adi 4 Veri 4
Listedeki hangi veriye sahip elemani silmek istersiniz?2
kullanici adi 1 Veri 1
kullanici adi 3 Veri 3
kullanici adi 4 Veri 4
Listedeki hangi veriye sahip elemani silmek istersiniz?1
kullanici adi 3 Veri 3
kullanici adi 4 Veri 4
Listedeki hangi veriye sahip elemani silmek istersiniz??4
kullanici adi 3 Veri 3
Listedeki hangi veriye sahip elemani silmek istersiniz?3

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\ciftyonludaireselbagliliste>

```

Çift yönlü dairesel bağlı bir
listede aradan düğümü
silen fonksiyonu gösterme

A) Çift Yönlü Dairesel Bağlı Liste (CDLL)

```
struct node {  
    std::string kullaniciadi;  
    int parola; // parola sayısal olarak tutuluyor  
    node* next;  
    node* prev;  
};
```

```
// 1. Parola toplamını güncelle  
void parolaGuncelleCDLL(node* bas, int delta) {  
    if (!bas) return;  
    node* iter = bas;  
    do {  
        iter->parola += delta;  
        iter = iter->next;  
    } while (iter != bas);  
}
```

```
// 3. Ters sırada yazdır  
void printReverseCDLL(node* bas) {  
    if (!bas) return;  
    node* iter = bas->prev; // sondan başlıyoruz  
    node* start = iter;  
    do {  
        std::cout << iter->kullaniciadi << ": " << iter->parola << std::endl;  
        iter = iter->prev;  
    } while (iter != start);  
}
```

```
// 6. Belirli uzunlukta isimleri yazdır  
void printNameLengthCDLL(node* bas, int length) {  
    if (!bas) return;  
    node* iter = bas;  
    do {  
        if (iter->kullaniciadi.length() == length)  
            std::cout << iter->kullaniciadi << ": " << iter->parola << std::endl;  
        iter = iter->next;  
    } while (iter != bas);  
}
```

```
// 2. Parola ortalaması  
double parolaOrtCDLL(node* bas) {  
    if (!bas) return 0;  
    node* iter = bas;  
    double toplam = 0;  
    int count = 0;  
    do {  
        toplam += iter->parola;  
        count++;  
        iter = iter->next;  
    } while (iter != bas);  
    return toplam / count;  
}
```

```
// 4. Çift parolaları yazdır
void printEvenCDLL(node* bas) {
    if (!bas) return;
    node* iter = bas;
    do {
        if (iter->parola % 2 == 0)
            std::cout << iter->kullaniciadi << ": " << iter->parola << std::endl;
        iter = iter->next;
    } while (iter != bas);
}
```

NOT

delete iter;

Bu noktada iter artık geçersiz (dangling pointer) olur.

Eğer iter = iter->next yazsaydın, iter artık silindiği için program çökebilirdi.

iter = nextNode;

Artık güvenle **bir sonraki düğüme geçiyorsun.**

```
// 5. Tek parolaları sil
node* removeOddCDLL(node* bas) {
    if (!bas) return NULL;
    node* iter = bas;
    node* start = bas;
    do {
        node* nextNode = iter->next;
        if (iter->parola % 2 != 0) {
            iter->prev->next = iter->next;
            iter->next->prev = iter->prev;
            if (iter == bas) bas = iter->next;
            delete iter;
        }
        iter = nextNode;
    } while (iter != start);
    return bas;
}
```

if (iter == bas)

bas = iter->next;

Bu kontrolün amacı:

Silinen düğüm listenin başlangıcıysa (bas), yeni başlangıcı güncellemek.

Dikkat Edilmesi Gereken Risk

Bu kodda küçük bir risk var: Eğer listede **tek bir düğüm** varsa ve o silinirse, döngü bittiğinde return bas; satırı silinmiş (hafızadan atılmış) bir adresi döndürebilir. Güvenli bir "tek düğüm" kontrolü normalde şöyle görünür:

```
if (iter->next == iter) {
    delete iter;
    return NULL;
}
```

Kaynakça

Veri Yapıları ve Algoritmalar, 12. Baskı, Rifat Çölkesen

Nejat Yumuşak, fatih Adak Veri C++ ile veri yapıları, 3.baskı

C++ ile programlama, Dokuzuncu baskıdan çeviri, Palme Yayınları, Pau Deitel, Harvey Deitel, Çeviri Editörü Cemil Öz

Veri Yapıları, Hakan Kutucu

Algoritma ve Programlamaya giriş, Fahri Vatansever

Programlama ve Veri Yapılarına Giriş C,C++ ve Java ile, Sadi Evren Şeker

<https://chatgpt.com/>